

Object References and Copying - Simple Hinglish Summary

Source: <https://javascript.info/object-copy>

Main baat

Objects aur primitives me ek **bunyadi fark** hai: **objects "by reference" store aur copy hote hain**, jabki primitive values (string, number, boolean, etc.) hamesha "**poori value ke roop me**" copy hoti hain.

1. Primitive ka copy

```
javascript  
  
let message = "Hello!";  
let phrase = message;
```

Nateeja: **do independent variables**, dono me ("Hello!") string. Ek badalne se dusra nahi badalta. Ye obvious hai.

2. Object alag hota hai

Object wale variable me object khud nahi hota, balki uska "**memory address**" (reference) hota hai.

```
javascript  
  
let user = {  
  name: "John"  
};
```

Object memory me kahin store hota hai, aur (user) variable me sirf uska **reference** hota hai. (user) ko ek **kagaz ke tukde** ki tarah socho jis par object ka **pata (address)** likha hai.

Jab hum (user.name) jaisa kuch karte hain, JS engine us address par jaakar asli object par kaam karta hai.

Copy karne par kya hota hai?

Object variable copy karne par sirf reference copy hota hai, object duplicate nahi hota.

```
javascript  
  
let user = { name: "John" };  
  
let admin = user; // reference copy hua
```

Ab **do variables** hain, lekin dono **ek hi object** ko point karte hain. Kisi bhi variable se object badal sakte ho:

```
javascript

let user = { name: 'John' };

let admin = user;

admin.name = 'Pete'; // "admin" reference se badla

alert(user.name); // 'Pete', "user" reference se bhi badlav dikhta hai
```

Analogy: Ek almari hai jiski do chabiyan hain. Ek chabi (**admin**) se kholkar badlav kiya, to dusri chabi (**user**) se kholne par bhi wahi badla hua saman milega.

3. Reference se comparison

Do objects tabhi barabar hote hain jab wo ek hi object ho.

```
javascript

let a = {};
let b = a; // reference copy

alert( a == b ); // true, dono ek hi object ko point karte hain
alert( a === b ); // true
```

Do **alag** objects barabar nahi hote, chahe dikhne me ek jaise hon:

```
javascript

let a = {};
let b = {}; // do independent objects

alert( a == b ); // false
```

(obj1 > obj2) ya (obj == 5) jaise comparisons me objects primitives me convert hote hain. Aise comparisons ki zarurat bahut kam padti hai, aur aksar wo programming ki galti hote hain.

const object badal sakta hai

Reference store hone ka ek zaruri side effect: **const** se declare kiya object modify ho sakta hai.

javascript

```
const user = {  
  name: "John"  
};  
  
user.name = "Pete"; // (*)  
  
alert(user.name); // Pete
```

Lagta hai line `(*)` par error aayega, lekin nahi aata. `user` ki value (reference) constant hai, yaani wo hamesha usi object ko point karega. Lekin us **object ki properties badal sakti hain**.

`const user` tabhi error deta hai jab poora `user = ...` reassign karne ki koshish karo.

(Properties ko bhi constant banana possible hai, lekin uske alag methods hain, jo "Property flags and descriptors" chapter me hain.)

4. Cloning aur merging: `Object.assign`

Object variable copy karne se sirf ek aur reference banta hai. To **object duplicate** karna ho to?

Tarika 1: Loop se haath se copy:

javascript

```
let user = {  
  name: "John",  
  age: 30  
};  
  
let clone = {}; // naya khali object  
  
// user ki saari properties usme copy karo  
for (let key in user) {  
  clone[key] = user[key];  
}  
  
// ab clone ek poori tarah independent object hai  
clone.name = "Pete";  
  
alert( user.name ); // abhi bhi John (original me nahi badla)
```

Tarika 2: `Object.assign`:

javascript

```
Object.assign(dest, ...sources)
```

- Pehla argument `dest` **target object** hai.
- Baaki arguments **source objects** ki list hai.
- Ye saare sources ki properties ko `dest` me copy karta hai aur `dest` ko hi return karta hai.

Example: `user` me kuch permissions jodna:

javascript

```
let user = { name: "John" };

let permissions1 = { canView: true };
let permissions2 = { canEdit: true };

Object.assign(user, permissions1, permissions2);

// ab user = { name: "John", canView: true, canEdit: true }
alert(user.name);    // John
alert(user.canView); // true
alert(user.canEdit); // true
```

Property ka naam pehle se ho to **overwrite** ho jata hai:

javascript

```
let user = { name: "John" };

Object.assign(user, { name: "Pete" });

alert(user.name); // ab user = { name: "Pete" }
```

Simple cloning ke liye:

javascript

```
let user = {
  name: "John",
  age: 30
};

let clone = Object.assign({}, user);

alert(clone.name); // John
alert(clone.age);  // 30
```

Khali object me `user` ki saari properties copy hoti hain aur wo return hota hai.

Aur bhi tarike hain, jaise **spread syntax**: `clone = {...user}` (ye aage seekhenge).

5. Nested cloning (object ke andar object)

Ab tak humne maana ki `user` ki saari properties primitive hain. Lekin properties **dusre objects ke references** bhi ho sakti hain:

```
javascript

let user = {
  name: "John",
  sizes: {
    height: 182,
    width: 50
  }
};

alert( user.sizes.height ); // 182
```

Ab `clone.sizes = user.sizes` kaafi nahi hai, kyunki `user.sizes` ek object hai aur wo **reference se copy** hoga. Isliye `clone` aur `user` **same** `sizes` share karenge:

```
javascript
```

```

let user = {
  name: "John",
  sizes: {
    height: 182,
    width: 50
  }
};

let clone = Object.assign({}, user);

alert( user.sizes === clone.sizes ); // true, same object

// user aur clone sizes share karte hain
user.sizes.width = 60;    // ek jagah se badla
alert(clone.sizes.width); // 60, dusri jagah se bhi dikha

```

Fix: clone ko sach me alag banane ke liye ek cloning loop chahiye jo `user[key]` ki har value dekhe, aur agar wo object ho to uska structure bhi replicate kare. Ise "**deep cloning**" ya "**structured cloning**" kehte hain. Iske liye `structuredClone` method hai.

`structuredClone`

`structuredClone(object)` object ko **saari nested properties ke saath** clone karta hai.

javascript

```

let user = {
  name: "John",
  sizes: {
    height: 182,
    width: 50
  }
};

let clone = structuredClone(user);

alert( user.sizes === clone.sizes ); // false, alag objects

// user aur clone ab पूरी तरह unrelated hain
user.sizes.width = 60;
alert(clone.sizes.width); // 50, related nahi

```

`structuredClone` zyadatar data types clone kar leta hai: objects, arrays, primitive values.

Circular references bhi support karta hai (jab object ki koi property object ko hi point kare):

javascript

```
let user = {};  
// circular reference: user.me khud user ko point karta hai  
user.me = user;  
  
let clone = structuredClone(user);  
alert(clone.me === clone); // true
```

Dekho: `clone.me`, `user` ko nahi balki `clone` ko point karta hai. Circular reference sahi se clone hua.

`structuredClone` kab fail hota hai?

Jab object me **function property** ho:

javascript

```
// error  
structuredClone({  
  f: function() {}  
});
```

Function properties support nahi hoti.

Aise complex cases ke liye hume alag cloning methods ka combination, custom code, ya existing implementation lagegi, jaise **lodash** library ka `_.cloneDeep(obj)`.

Summary

- **Objects reference se assign aur copy hote hain.** Variable me "object ki value" nahi, balki uska "reference" (memory address) hota hai. Isliye aise variable ko copy karna ya function argument me pass karna **reference copy karta hai, object nahi.**
- Copy hue references se kiye gaye saare operations (properties add/remove karna) **usi ek object** par hote hain.
- **Asli copy (clone)** banane ke liye:
 - `Object.assign` → **shallow copy** (nested objects reference se copy hote hain)
 - `structuredClone` → **deep cloning** (nested objects bhi copy)
 - Ya custom implementation, jaise `_.cloneDeep(obj)` (lodash)

Quick Summary

Baat	Yaad rakho
Primitive copy	Poori value copy hoti hai (independent)
Object copy	Sirf reference copy hota hai (dono ek hi object)
<code>a == b</code> (objects)	Tabhi <code>true</code> jab ek hi object ho
<code>const obj</code>	Object ki properties badal sakti hain, sirf reassign nahi
Shallow copy	<code>Object.assign({}, obj)</code> ya <code>{...obj}</code>
Deep copy	<code>structuredClone(obj)</code>
<code>structuredClone</code> limit	Functions wali properties par fail hota hai
Function wale objects	<code>_.cloneDeep(obj)</code> (lodash)

Yaad rakho: Object copy karna ho to `let b = a;` **kaafi nahi hai**. Wo sirf ek aur reference banata hai. Asli alag copy ke liye `Object.assign` ya `structuredClone` use karo.